

Lab 3: Component Modelling & Architectural Pattern Selection

Name: Sripada Krishna Sameer Aditya

SRN: PES1UG24AM285

1. Architecture Selection

We chose Layered Architecture for the Campus Club Discussion & Meetup System.

The system separates the user-facing screens, business logic (topic/meetup handling, RSVP/waitlist handling, authentication, notifications), and data storage into distinct layers, which matches the way the components in the design are organised and communicate through defined APIs. This layered separation keeps the presentation layer independent of how meetups, RSVPs, and notifications are processed, while all layers ultimately rely on a single shared database layer.

2. Architectural Style Analysis

Layered Architecture (Selected): The system is organised into a presentation layer (User Interface), a business logic layer (Authentication, Topic/Meetup Management, RSVP/Waitlist Management, Notification Service), and a data layer (Database). This gives a clear separation of concerns and is easy to reason about for a system with a fixed, well-defined set of responsibilities.

Microservices Architecture: Not selected, as it introduces independent deployment and network complexity that is unnecessary here — all components share a single database and are tightly related in workflow (topic approval, RSVP, notification), which is better suited to layered communication than independently scaled services.

Client-Server Architecture: Not selected on its own, as it only describes the split between the client (UI) and a server, without addressing the internal separation of authentication, meetup/RSVP logic, and notifications that this system requires. Layered architecture provides this internal structure while still following an overall client-server style at the UI-to-backend level.

3. Component Identification

- **User Interface** – Provides the screens for browsing/proposing topics, submitting RSVPs, scheduling meetups, approving/rejecting proposed topics, and viewing attendance.
- **Authentication Service** – Authenticates user credentials and enforces role validation and session/token checks at the UI and API boundary.
- **Topic & Meetup Management** – Handles topic proposals, approval/rejection, and meetup scheduling and detail management.
- **RSVP & Waitlist Management** – Handles RSVP submissions, capacity checking, automatic waitlisting, and promotion from the waitlist on cancellations.
- **Database** – Stores members, leads, topics, meetups, capacity, RSVPs, waitlist entries, attendance, and notifications.
- **Notification Service** – Sends confirmations, waitlist/promotion notices, meetup scheduling updates, and topic decisions to users.

4. Interface and Component Interactions

- **Auth API** – User Interface calls the Authentication Service to authenticate credentials.
- **Topic/Meetup API** – User Interface sends topic proposals and scheduling requests to Topic & Meetup Management.
- **RSVP API** – User Interface submits RSVPs and receives waitlist status from RSVP & Waitlist Management.
- **Topic/Meetup DB Queries** – Topic & Meetup Management stores and retrieves meetups and topics from the Database.
- **RSVP DB Queries** – RSVP & Waitlist Management stores RSVPs, capacity, and waitlist data in the Database.

- Notification API (Includes RSVP Notifications) – Both Topic & Meetup Management and RSVP & Waitlist Management send meetup scheduling, topic decision, RSVP confirmation, waitlist, and promotion events to the Notification Service.
- Notification Service also reads/stores meetup, RSVP, waitlist, and notice details via the Database.

5. Security Consideration

The Authentication Service is isolated as its own layer and enforces role validation and session/token checks at the UI and API boundary (as shown in the diagram). This means every request from the User Interface must pass through authentication before reaching the Topic/Meetup or RSVP/Waitlist layers, preventing unauthorised users from proposing topics, scheduling meetups, or submitting RSVPs on behalf of others.

6. Performance Consideration

Separating RSVP/Waitlist Management from Topic/Meetup Management allows RSVP submissions and capacity checks (a frequent, time-sensitive operation) to be processed without being slowed down by topic approval or scheduling logic. Because both layers query the same Database layer independently, RSVP status and waitlist promotion can be handled quickly even when meetup scheduling activity is high.

7. UML Component Diagram

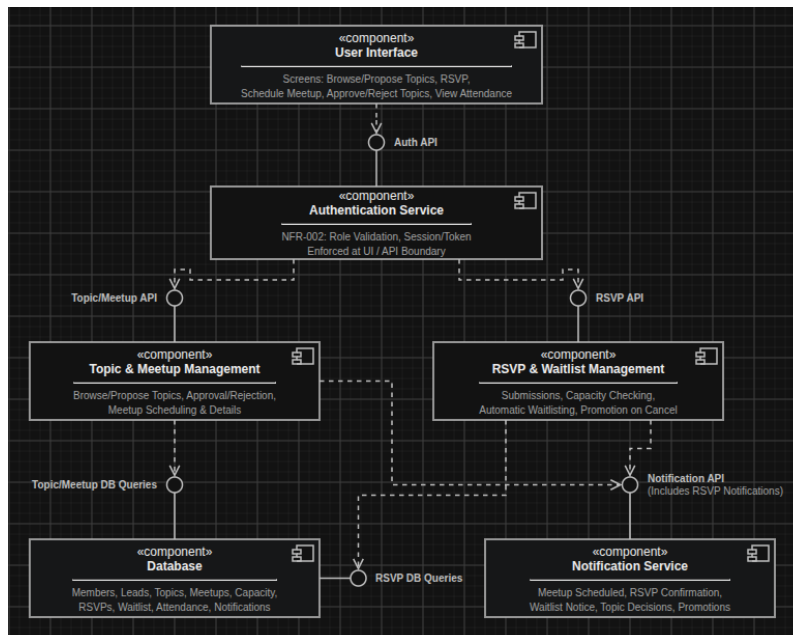


Figure 1: UML Component Diagram of the Campus Club Discussion & Meetup System